第9章 结构体与共用体

9.1 学习目标

- 掌握结构体的定义与使用
- 理解结构体数组
- 掌握结构体指针
- 了解共用体和枚举类型

9.2 为什么需要结构体

```
// 没有结构体：管理学生信息很麻烦
char name1[20];  int age1;  float score1;
char name2[20];  int age2;  float score2;
// ... 数据和数据之间的关联不够直观

// 有结构体：将相关数据组织在一起
struct Student {
    char name[20];
    int age;
    float score;
};
// 一个变量就包含了学生的所有信息，清晰直观！
```

> 结构体 = 用户自定义的复合数据类型，将不同类型的数据组合成一个整体。

9.3 结构体的定义

9.3.1 三种定义方式

```
// 方式1：先定义结构体类型，再定义变量（推荐）
struct Student
{
    char name[20];
    int age;
    float score;
};
struct Student s1, s2;    // 用 struct Student 类型定义变量

// 方式2：定义类型的同时定义变量
struct Student
{
    char name[20];
    int age;
    float score;
```

```
} s1, s2;

// 方式3：匿名结构体（不推荐，只能用一次）
struct
{
    char name[20];
    int age;
} s1, s2;
```

9.3.2 typedef 简化类型名

```
typedef struct
{
    char name[20];
    int age;
    float score;
} Student;            // Student 成为类型别名

Student s1, s2;      // 不用写 struct 关键字了
Student class[50];    // 结构体数组
```

9.4 结构体变量的使用

9.4.1 成员访问（.运算符）

```
#include <stdio.h>
#include <string.h>

typedef struct
{
    char name[20];
    int age;
    float score;
} Student;

int main()
{
    Student s1;

    // 逐个成员赋值
    strcpy(s1.name, "张三");
    s1.age = 20;
    s1.score = 88.5;

    // 成员访问
    printf("姓名: %s\n", s1.name);
    printf("年龄: %d\n", s1.age);
    printf("成绩: %.1f\n", s1.score);

    return 0;
```

```
}
```

9.4.2 整体赋值(C允许同类型结构体直接赋值)

```
Student s1 = {"张三", 20, 88.5};  // 初始化
Student s2;

s2 = s1;   // ? 同类型结构体可以整体赋值
// s2 的每个成员都被复制为 s1 成员的值
```

9.4.3 结构体输入输出

```
#include <stdio.h>

typedef struct
{
    char name[20];
    int age;
    float score;
} Student;

void inputStudent(Student *s)
{
    printf("姓名: ");  scanf("%s", s->name);
    printf("年龄: ");  scanf("%d", &s->age);
    printf("成绩: ");  scanf("%f", &s->score);
}

void printStudent(const Student *s)
{
    printf("姓名: %s, 年龄: %d, 成绩: %.1f\n",
           s->name, s->age, s->score);
}

int main()
{
    Student s1;
    inputStudent(&s1);   // 传地址
    printStudent(&s1);
    return 0;
}
```

9.5 结构体数组

```
#include <stdio.h>

typedef struct
{
    char name[20];
```

```
    int age;
    float score;
} Student;

int main()
{
    // 定义并初始化结构体数组
    Student class[3] = {
        {"张三", 20, 88.5},
        {"李四", 21, 92.0},
        {"王五", 19, 76.3}
    };

    // 遍历
    for (int i = 0; i < 3; i++)
    {
        printf("%s %d %.1f\n",
               class[i].name,
               class[i].age,
               class[i].score);
    }

    // 找最高分
    int maxIndex = 0;
    for (int i = 1; i < 3; i++)
    {
        if (class[i].score > class[maxIndex].score)
            maxIndex = i;
    }
    printf("最高分: %s %.1f\n",
           class[maxIndex].name, class[maxIndex].score);

    return 0;
}
```

9.6 结构体指针

9.6.1 指向结构体的指针

```
Student s = {"张三", 20, 88.5};
Student *p = &s;    // p指向s

// 三种等价方式访问成员
s.age           // 变量名.成员
(*p).age        // 解引用后.成员（括号不能少，.优先级高于*）
p->age          // 指针->成员（最常用，等价于上面）
```

9.6.2 箭头运算符 `->`

```
typedef struct { int x; int y; } Point;
```

```c
void movePoint(Point *p, int dx, int dy)
{
    p->x += dx;    // 等价于 (*p).x += dx
    p->y += dy;
}

int main()
{
    Point pt = {10, 20};
    movePoint(&pt, 5, -3);
    printf("(%d, %d)\n", pt.x, pt.y);  // (15, 17)
    return 0;
}
```

9.6.3 结构体数组排序

```c
#include <stdio.h>
#include <string.h>

typedef struct
{
    char name[20];
    float score;
} Student;

// 按成绩排序
void sortByScore(Student arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (arr[j].score < arr[j + 1].score)  // 降序
            {
                Student temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main()
{
    Student class[5] = {
        {"Alice", 88.5}, {"Bob", 92.0}, {"Charlie", 76.3},
        {"David", 85.0}, {"Eve", 95.5}
    };

    sortByScore(class, 5);
```

```
    printf("排名  姓名    成绩\n");
    for (int i = 0; i < 5; i++)
        printf("%-5d %-8s %.1f\n", i+1, class[i].name, class[i].score);

    return 0;
}
```

9.7 共用体（Union）

9.7.1 概念

> 共用体：所有成员共享同一块内存空间，同一时间只能存储一个成员的值。

```
#include <stdio.h>

union Data
{
    int i;
    float f;
    char str[20];
};

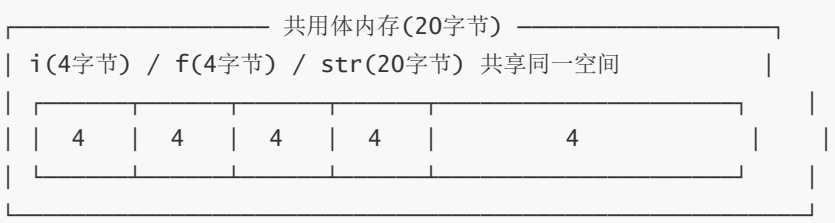
int main()
{
    union Data data;

    printf("共用体大小: %d 字节\n", sizeof(data));  // 20（最大成员的大小）

    data.i = 10;
    printf("data.i = %d\n", data.i);   // 10

    data.f = 3.14;                      // 覆盖了i的值!
    printf("data.f = %f\n", data.f);    // 3.14
    printf("data.i = %d\n", data.i);    // 乱码（已被覆盖）

    return 0;
}
```



9.7.2 应用场景

```
// 学生/教师信息：根据角色使用不同字段
typedef struct
{
```

```
    char name[20];
    int role;         // 0=学生, 1=教师
    union
    {
        float score;    // 学生：成绩
        char title[20]; // 教师：职称
    } info;
} Person;

int main()
{
    Person p1 = {"张三", 0};
    p1.info.score = 88.5;   // 学生用score

    Person p2 = {"李老师", 1};
    strcpy(p2.info.title, "教授");  // 教师用title

    if (p1.role == 0)
        printf("学生 %s 成绩: %.1f\n", p1.name, p1.info.score);
    else
        printf("教师 %s 职称: %s\n", p2.name, p2.info.title);

    return 0;
}
```

9.8 枚举类型（Enum）

```
// 定义枚举类型
enum WeekDay { MON, TUE, WED, THU, FRI, SAT, SUN };
// 默认值: MON=0, TUE=1, WED=2, ...

enum Season { SPRING = 1, SUMMER, AUTUMN, WINTER };
// 指定起始值: SPRING=1, SUMMER=2, AUTUMN=3, WINTER=4

// 实际用途
#include <stdio.h>

typedef enum { RED, YELLOW, GREEN } TrafficLight;

void action(TrafficLight light)
{
    switch (light)
    {
        case RED:    printf("停车\n"); break;
        case YELLOW: printf("减速\n"); break;
        case GREEN:  printf("通行\n"); break;
    }
}

int main()
{
```

```
    TrafficLight light = RED;
    action(light);  // 停车

    light = GREEN;
    action(light);  // 通行

    return 0;
}
```

> 枚举让代码更具可读性：`RED` 比 `0` 更好理解。

9.9 综合案例：学生成绩管理系统

```
#include <stdio.h>
#include <string.h>
#define MAX 100

typedef struct
{
    int id;              // 学号
    char name[20];       // 姓名
    float math;          // 数学
    float english;       // 英语
    float cProgramming;  // C语言
    float total;         // 总分
    float average;       // 平均分
} Student;

// 计算总分和平均分
void calcScore(Student *s)
{
    s->total = s->math + s->english + s->cProgramming;
    s->average = s->total / 3.0;
}

// 输入学生信息
void inputStudents(Student arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("\n第%d个学生:\n", i + 1);
        printf("学号: "); scanf("%d", &arr[i].id);
        printf("姓名: "); scanf("%s", arr[i].name);
        printf("数学 英语 C语言: ");
        scanf("%f %f %f", &arr[i].math, &arr[i].english, &arr[i].cProgramming);
        calcScore(&arr[i]);
    }
}

// 打印成绩单
void printReport(const Student arr[], int n)
```

```
{
    printf("\n====== 成绩单 ======\n");
    printf("%-6s %-10s %-6s %-6s %-6s %-7s %-6s\n",
           "学号", "姓名", "数学", "英语", "C语言", "总分", "平均");
    printf("-----------------------------------------------\n");

    for (int i = 0; i < n; i++)
    {
        printf("%-6d %-10s %-6.1f %-6.1f %-6.1f %-7.1f %-6.1f\n",
               arr[i].id, arr[i].name,
               arr[i].math, arr[i].english, arr[i].cProgramming,
               arr[i].total, arr[i].average);
    }
}

// 按总分排序（冒泡）
void sortByTotal(Student arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (arr[j].total < arr[j + 1].total)  // 降序
            {
                Student temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main()
{
    Student class[MAX];
    int n;

    printf("请输入学生人数: ");
    scanf("%d", &n);

    inputStudents(class, n);
    sortByTotal(class, n);
    printReport(class, n);
    printf("\n");

    return 0;
}
```

9.10 本章小结

知识点	要点
结构体定义	`struct 结构体名 { 成员列表 };`
成员访问	变量用 `.`，指针用 `->`
typedef	`typedef struct { ... } 别名;`
结构体数组	`Student arr[50];`
结构体赋值	同类型可直接 `=` 赋值
共用体	成员共享内存，大小为最大成员
枚举	`enum 名称 { 常量列表 };` 提高代码可读性

9.11 课后作业

基础练习

1. 选择题

(1) 设有 `struct { int x; int y; } s[2] = {{1,3},{2,7}};` 则 `s[0].y / s[1].x` 的值为（　）：

A. 0　　B. 1　　C. 2　　D. 3

(2) 结构体指针访问成员使用（　）：

A. `.`　　B. `->`　　C. `*`　　D. `&`

2. 判断题

(1) 共用体变量同一时间可以存储所有成员的值。（　）

(2) 枚举类型的值默认从0开始递增。（　）

编程练习

3. 日期结构体

定义 `Date` 结构体（年、月、日），编写函数判断某日期是该年的第几天。

4. 复数运算

定义 `Complex` 结构体（实部、虚部），实现复数加减乘运算。

5. 通讯录管理

定义 `Contact` 结构体（姓名、电话、邮箱），实现添加、查找（按姓名）、显示全部功能。

6. 矩形与圆

用共用体设计一个数据结构，既能表示矩形（长、宽），也能表示圆（半径），计算面积。

7. 学生成绩排名（综合）

定义 `Student` 结构体（学号、姓名、三门课成绩、总分、平均分、名次），输入n个学生信息，按总分排名并输出名次表。

> **学习建议**：结构体是面向对象编程中"类"的基础。学好结构体，后续学习 C++/Java 会事半功倍。